

## Phase 1: Frontend Foundation & UI/UX Implementation

**Goal:** Build out the complete, pixel-perfect user interface using static mock data so stakeholders can click through the flow before the database is connected.

- **Step 1: Project Initialization & Theming**
  - Scaffold the React project using Vite.
  - Install dependencies: Tailwind CSS, react-router-dom, lucide-react (for icons), and recharts (for charts).
  - Configure tailwind.config.js with the dark theme color palette (Slate Blue, Vivid Blue, Emerald Green).
- **Step 2: Core Layout & Navigation**
  - Build the persistent left <Sidebar /> with active/inactive states.
  - Create the main layout wrapper to handle the overall screen structure.
- **Step 3: Reusable Component Library**
  - Develop form controls: <FormInput />, <FormSelect /> with standardized Tailwind styling.
  - Build UI elements: <Button /> (Primary, Secondary, Success), <StatCard />, and the <Stepper /> component for the multi-step form.
  - Build a robust <DataTable /> component to handle the "Recent Cases" and "Past Analysis" views.
- **Step 4: Screen Construction (with Mock Data)**
  - **Dashboard:** Assemble stat cards, mock charts, and the recent cases table.
  - **New Analysis Flow (Steps 1 & 2):** Build the forms and the measurement guide image panel.
  - **Prediction Results (Step 3) & Report:** Build the results layout, confidence score UI, and the "Similar Cases" section.
  - **Past Analysis:** Implement the full-page data table view.

## Phase 2: Database & Backend Foundation

**Goal:** Set up the data layer and the API needed to make the UI functional.

- **Step 1: Database Setup (Supabase)**
  - Create the Supabase project.
  - Establish tables based on the schema: users, cases, measurements, and predictions.
  - Insert sample data into Supabase to test API routes.
- **Step 2: Backend Initialization (Node.js/Express)**
  - Initialize the Node/Express server.
  - Set up CORS and Supabase client SDK in the backend.
- **Step 3: API Endpoint Development**
  - Create GET /dashboard/stats for the main dashboard.
  - Create GET /cases (with pagination and search logic) for the Past Analysis screen.
  - Create POST /analysis/predict to handle the incoming form data and save it to the

database.

## Phase 3: State Management & System Integration

**Goal:** Connect the static React frontend to the live Express API.

- **Step 1: Multi-Step Form Logic**
  - Implement React Context or Zustand to hold the user's input from Step 1 so it isn't lost when moving to Step 2.
- **Step 2: API Integration**
  - Replace frontend mock data with axios or fetch calls to your Node.js backend.
  - Connect the Dashboard charts to real data.
  - Hook up the "New Analysis" submission to send the context data to the backend.
- **Step 3: Interactive Features**
  - Implement functional server-side pagination and search filtering on the Past Analysis table.
  - Add loading states (spinners) and error handling (toast notifications) for API calls.

## Phase 4: Advanced Features & Polish

**Goal:** Add the final technical requirements and prepare for production.

- **Step 1: Report Generation**
  - Implement PDF export functionality on the Prediction Report screen using a library like html2pdf.js or jspdf.
- **Step 2: ML Model Connection (If Applicable)**
  - Connect the Express backend to the actual Python/ML prediction script or microservice. (If the ML model isn't ready, finalize a robust mock response mechanism).
- **Step 3: QA & Bug Fixing**
  - Test across different screen sizes (ensure it degrades gracefully on smaller desktop windows).
  - Verify all form validation works as expected.

---

Since Phase 1 focuses entirely on the UI/UX, would you like me to start by generating the React/Tailwind code for the foundational layout and Sidebar components?